Università degli Studi di Trento Corso di Programmazione I – E3: 145430/145935 5° Appello – 04/09/2019 – a.a. 2018/2019	**PC N°**
COGNOME ..... **NOME .....** **MATRICOLA** | | | | | | |	
RITIRATO ☐ **FIRMA PER RITIRO.....**	

Completare le voci **Cognome**, **Nome** e **Matricola**, se tali voci saranno mancanti o non-leggibili elaborato **NON** verrà corretto.
NON è consentito utilizzo di alcun strumento cartaceo, elettronico, informatico etc. durante la prova.
Fa parte della valutazione la leggibilità del codice C/C++.
Scrivere le risposte su questo foglio e riconsegnare. I risultati verranno pubblicati su DOL
Tempo a disposizione: 120 minuti.

PARTE TEORICA: 13 PUNTI – PUNTEGGIO MINIMO 6 PUNTI

[Domanda 1 – punti 4] Risolvere la seguente equazione di ricorrenza, specificando il limite asintotico superiore.

$$\begin{cases} T(1) = 1 \\ T(N) = 2*T(N/2)+1 \end{cases}$$

[Domanda 2 - punti 2] Scrivere una funzione ricorsiva che calcola il Massimo Comune Divisore (MCD)di due numeri interi m ed n in base alle seguenti proprietà:
caso m<n, allora MCD(m,n)=MCD(m,n-m)
caso m==n, allora MCD(m,n)=m
caso m>n, allora MCD(m,n)=MCD(n,m-n)

[Domanda 3 – punti 2] Calcolare la somma tra $(B2)_{16}$ e $(3B)_{16}$.
Mostrare tutti i passaggi! Inoltre dire a quali delle soluzioni equivale il risultato
SI ☐ NO ☐ $(11101001)_2$
SI ☐ NO ☐ $(DE)_{16}$
SI ☐ NO ☐ $(355)_8$
SI ☐ NO ☐ $(237)_{10}$

[Domanda 4 - punti 4] Data la seguente struttura:

```
typedef struct Tnodo{
  int dato;
  Tnodo *next;
} Tnodo;
```

scrivere una funzione che verificare che il la lista contenga dati ordinati in <u>modo decrescente</u>. Definire e creare una funzione a cui viene passato in modo opportuno la struttura e che restituisce 1 se è ordinato e 0 se non lo è.

[Domanda 5 - punti 1] Dopo l'esecuzione di

```
int c; int x = 2; float y = 1.5; c = x/y;
```

quanto vale c?
SI ☐ NO ☐ 1
SI ☐ NO ☐ 4/3
SI ☐ NO ☐ 2
SI ☐ NO ☐ nessuna delle precedenti.

PARTE PRATICA: 20 PUNTI – PUNTEGGIO MINIMO 10 PUNTI

[A - punti 3] *Inserire i seguenti tipi in un file* ***dati.h*** **e implementare i metodi di stampa ed i costruttori:**

```
typedef enum Tproblema {TECNICO,AMMINISTRATIVO,COMMERCIALE} Tproblema;
typedef struct Trichiesta {
  char IDCliente[10];
  unsigned int tempoAttesa;
  Tproblema tipoProblema;
  Trichiesta ()  { /*  da implementare */ }
  void stampa() { /*  da implementare → formato: vedi punto E */ }
} Trichiesta;
typedef struct Tnodo {
   Trichiesta dato;
   Tnodo* next;
   Tnodo () { /*  da implementare */ }
   Tnodo (Trichiesta a, Tnodo* n) { /*  da implementare */ }
   void stampa() { /*  da implementare */ }
} Tnodo;
```

[B - punti 1] *Modificare il file* ***main.cpp*** *includendo il file* ***dati.h*** *ed inserendo il seguente codice nel* ***main****:*

```
Tnodo* listaTicket = NULL;
Trichiesta r;
for(int i=0; i<10; i++) {
    newRichiesta(&r); listaTicket = addRichiesta(listaTicket, r); }
while(listaTicket != NULL) { listaTicket = gestioneTicket(listaTicket); }
```

[C - punti 3] Implementare una funzione **newRichiesta** che riceve in input l'indirizzo di una variabile di tipo **Trichiesta r** e la inizializza con:

- campo **IDCliente** letta da tastiera;
- campo **tempoAttesa** valore pari a 1;
- campo **tipoProblema** valore casuale tra TECNICO,AMMINISTRATIVO,COMMERCIALE.

[D - punti 4] Implementare una funzione **addRichiesta** che riceve come input una variabile di tipo **Tnodo*** **lt** e una di tipo **Trichiesta r** ed inserisce la richiesta nella listaFIFO (**insertLast**). La funzione restituisce il puntatore alla testa della lista.

[E - punti 2] Implementare una funzione **stampaTicket** che riceve in input una variabile di tipo **Tnodo*** **lt** e stampa il contenuto della lista FIFO utilizzando opportunamente il metodo **stampa di Tnodo**. Nella forma (per l'enum stampare etichetta in modo opportuno):

```
tipoProblema (IDCliente) Tempo= tempoAttesa
```

[F - punti 7] Implementare una funzione **gestioneTicket** che riceve come input una variabile di tipo **Tnodo*** **lt** e: (1) invoca la funzione **stampaTicket**; (2) rimuove l'elemento in testa (**removeFirst**); (3) genera un valore casuale tra 0 e 1; (4) se valore pari a 0 re-inserisce l'elemento nella FIFO (**insertLast**); (5) se valore pari a 1 aggiunge la richiesta ad un file denominato **"ticketGestiti.txt"** nella forma:

```
tipoProblema (IDCliente) Tempo= tempoAttesa
```

(6) scorre tutta la lista ed incrementa il campo tempoAttesa di 1.
La funzione restituisce il puntatore alla testa della lista.

È possibile creare strutture, funzioni, metodi aggiuntivi per realizzare gli esercizi proposti.
Il codice commentato NON verrà corretto!